BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 1

Roll no:51 PRN No:12412161 Name of Student:Utkarsh Salunkhe

Subject Name : Deep Learning

Problem Statement : Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

1. **Import Libraries**

- Import TensorFlow.
- Import Matplotlib for data visualization.

2. **Load Dataset**

- Load the MNIST dataset using tf.keras.datasets.mnist.
- Split the dataset into training and testing sets.

3. **Preprocess Data**

- Normalize pixel values by dividing each pixel by 255.
- Convert pixel values from the range 0–255 to 0–1.

4. **Visualize Data**

- Display the shape of training and testing datasets.
- Print training labels.
- Display sample handwritten digit images.

5. **Build the Neural Network**

- Create a Sequential model.
- Add a Flatten layer to convert the 28×28 image into a 1D vector.
- Add a Dense hidden layer with ReLU activation.

- Add an Output Dense layer with 10 neurons and Softmax activation for digit classification (0–9).

6. Compile the Model

- Use the Adam optimizer.
- Use Sparse Categorical Crossentropy as the loss function.
- Use Accuracy as the evaluation metric.

7. Train the Model

- Train the neural network using the training dataset.
- Run training for the specified number of epochs.

8. Evaluate the Model

- Test the trained model on the testing dataset.
- Calculate and display the test accuracy and loss.

9. Save the Model

- Save the trained model for future prediction using my_model.save().

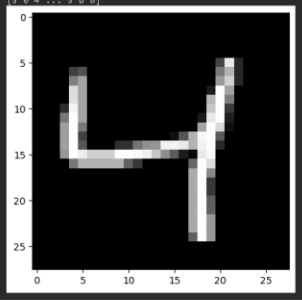
10. End

- The trained model is ready to classify handwritten digits from the MNIST dataset.

Code :



```
(60000, 28, 28)
(10000, 28, 28)
[5 0 4 ... 5 6 8]
```



```
# Defining neural network model
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))     #Rectified Linear Unit        #Dense layers are used to learn complex patterns in data.
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))   #softmax is often used as the activation function in the output layer for classification tasks

# Compiling the model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) #Adaptive Moment Estimation #sparse_categorical_crossentropy for multi-class classification
```

```
# Compiling the model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) #Adaptive Moment Estimation #sparse_categorical_crossentropy for multi-class classification

# Training the model
my_model.fit(train_images, train_labels, epochs=10)

Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - accuracy: 0.9255 - loss: 0.2577
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9663 - loss: 0.1138
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9760 - loss: 0.0765
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9824 - loss: 0.0570
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9862 - loss: 0.0450
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9892 - loss: 0.0349
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9914 - loss: 0.0275
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9934 - loss: 0.0213
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9940 - loss: 0.0195
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9953 - loss: 0.0157
<keras.src.callbacks.history.History at 0x7b178d67d5b0>
```

```
# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)

313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9786 - loss: 0.0867
Test accuracy:  0.978600025177002
```

Output :

```
my_model.save('my_mnist_model.keras')
```

```
my_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_2 (Dense)	(None, 128)	100,480
dense_3 (Dense)	(None, 10)	1,290

Total params: 305,312 (1.16 MB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 203,542 (795.09 KB)